Machine Learning For Networking

Stridi Simone

Anno Accademico 2026/2027

Contents

1 Data PreProcessing **3**
1.1 Data Types and Properties 3
1.1.1 Data Quality 4
1.2 Similarity and Dissimilarity in Data 4
1.2.1 Similarity and Dissimilarity for Simple Attributes 4
1.2.2 Similarity and Dissimilarity for Structured Attributes 5
1.2.3 Properties of a Distance Metric 7
1.2.4 Mahalanobis Distance 8
1.3 Data PreProcessing 9
1.3.1 Curse of Dimensionality 11
1.3.2 Feature Engineering 13
1.3.3 Data Transformation 13
1.3.4 Data Discretization 14
1.3.5 Data Binarization 14
1.3.6 Attribute Transformation 15
1.3.7 Data Preparation for Document Data 16

Chapter 1

Data PreProcessing

1.1 Data Types and Properties

Attribute values are assigned to an attribute for a particular object. It is useful to remember the difference between attributes and attribute values, as the same attribute can be mapped to different attribute values (for instance, height can be measured in feet or meters). Moreover, different attributes can be mapped to the same set of values (for example, both ID and age can be integers).

There are 2 main attribute types:

- **Categorical (Nominal):** for instance ID numbers, eye color.
- **Ordinal:** for instance time, count, grades. A set of ordered attribute values.

The main difference between these 2 attribute types resides in the properties of the corresponding attribute values:

- **Nominal (Categorical) Attribute Property:** Distinctness, operator (=).
- **Ordinal Attribute Properties:** Distinctness and Order, operators ($<$, $=$, $>$).

Moreover, there are Discrete and Continuous Attributes:

- **Discrete Attribute:** finite set of values, often represented as integer variables. Binary attributes are a special case of discrete attributes.
- **Continuous Attribute:** real numbers as attribute values. In practice, real values can only be measured and represented using a finite number of digits.

Regarding Data Types we have:

- **Tabular Data:** collection of records, each of them characterized by a fixed set of attributes.
- **Document Data:** semi-structured or unstructured textual data.
- **Graph Data:** for instance the World Wide Web.
- **Ordered Data:** for instance spatio-temporal data or genomic sequences.

Tipo di attributo	Proprietà / operatori	Esempi
Nominale (categorico)	Distinzione (=)	ID, colore degli occhi
Ordinale	Distinzione e ordine (<, =, >)	Voti, taglie S/M/L
Discreto	Insieme finito, spesso interi	Conteggi, binari (0/1)
Continuo	Numeri reali (cifre finite)	Altezza, temperatura
Tipi di dati		
Tabellari	Record con attributi fissi	Dataset clienti
Documenti	Testo semi/non strutturato	Articoli, email
Grafi	Nodi e collegamenti	World Wide Web
Ordinati	Sequenze spazio-temporali	Genoma, serie storiche

Table 1.1: Riepilogo di attributi e tipi di dati

1.1.1 Data Quality

Each dataset comes with data quality issues, represented by:

- **Noise and Outliers:** Noise refers to the modification of original values, for instance noise can be introduced in the data recording process. It is important to note that noise is considered to be randomly distributed. On the other hand, outliers are data objects with characteristics that are considerably different from most of the other data in the dataset. Outliers can be noise that interferes with the data analysis, or they can also be the goal of the data analysis.
- **Missing Values:** either the information is not collected or some attributes may not be applicable to all cases. Missing values must be handled by eliminating them, estimating them or ignoring them during the analysis.
- **Duplicate Data:** the dataset may include data objects that are duplicates, or almost duplicates, of one another. It is a major issue when merging data from heterogeneous sources, and Data Cleaning is needed.

1.2 Similarity and Dissimilarity in Data

Similarity is the numerical measure of how alike two data objects are. It is a metric that often falls in the $[0, 1]$ range and is higher when objects are more alike. **Dissimilarity** is the numerical measure of how different 2 data objects are. It is lower when objects are more alike and is often a **distance metric**.

1.2.1 Similarity and Dissimilarity for Simple Attributes

A **dissimilarity** $d(x, y)$ is a numerical measure of how *different* two data objects are: it is 0 when the objects are identical and grows as they become more different. A **similarity** $s(x, y)$ measures how *alike* two objects are: it is usually bounded in $[0, 1]$, with 1 meaning identical. The two notions are complementary, so similarity is typically obtained by transforming dissimilarity.

For simple attributes (1 dimension), the appropriate measure depends on the attribute type, since each type supports different operations:

Attribute Type	**Dissimilarity**	**Similarity**
Nominal	$d = 0$ if $x = y$; $d = 1$ if $x \neq y$	$s = 1$ if $x = y$; $s = 0$ if $x \neq y$
Ordinal	$d = \frac{\lvert x - y \rvert}{n - 1}$ (values mapped to integers 0 to $n - 1$, where n is the number of values)	$s = 1 - d$
Interval or Ratio	$d = \lvert x - y \rvert$	$s = -d$, $s = \frac{1}{1+d}$, $s = e^{-d}$, $s = 1 - \frac{d - \min d}{\max d - \min d}$

Nominal attributes. Only the equality operator is meaningful, so two values are either identical or different: there is no notion of "more different". For example, $d(\text{red}, \text{red}) = 0$, while $d(\text{red}, \text{blue}) = d(\text{red}, \text{green}) = 1$.

Ordinal attributes. Values have an order, so some pairs are farther apart than others. The values are first mapped to consecutive integers $0, \ldots, n-1$, and the difference is divided by $n-1$ so that $d \in [0, 1]$. For example, with $\{\text{poor}, \text{fair}, \text{good}, \text{excellent}\}$ we have $n = 4$ and the mapping $\{0, 1, 2, 3\}$:

$$d(\text{poor}, \text{good}) = \frac{|0 - 2|}{3} = \frac{2}{3}, \qquad s = 1 - \frac{2}{3} = \frac{1}{3}.$$

Note that this approach implicitly assumes that consecutive values are equally spaced, which is not always true.

Interval and ratio attributes. The natural dissimilarity is the absolute difference $d = |x - y|$, which ranges in $[0, +\infty)$. Different transformations turn it into a similarity, all decreasing as d increases:

- $s = -d$: simplest choice, but not bounded in $[0, 1]$;
- $s = \frac{1}{1+d}$: ranges in $(0, 1]$, with $s = 1$ when $d = 0$;
- $s = e^{-d}$: ranges in $(0, 1]$ and decreases faster;
- $s = 1 - \frac{d - \min d}{\max d - \min d}$: min-max normalization over the dataset, ranges in $[0, 1]$.

For example, with $d = 2$ we obtain $s = -2$, $s = \frac{1}{3} \approx 0.33$ and $s = e^{-2} \approx 0.14$ respectively.

1.2.2 Similarity and Dissimilarity for Structured Attributes

When data objects are described by more than one attribute, each object can be seen as a point (or vector) in an n-dimensional space, where n is the number of attributes. Throughout this section, $\mathbf{x} = (x_1, \ldots, x_n)$ and $\mathbf{y} = (y_1, \ldots, y_n)$ denote two data objects, and x_k, y_k are their k-th attributes.

- **Euclidean Distance:** the length of the straight segment connecting $\mathbf{x}$ and $\mathbf{y}$. It generalizes the Pythagorean theorem to n dimensions: the differences along each attribute are squared, summed, and the square root is taken.

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{k=1}^{n} (x_k - y_k)^2}$$

  For example, with $\mathbf{x} = (1, 1)$ and $\mathbf{y} = (4, 5)$ the differences are 3 and 4, so $d(\mathbf{x}, \mathbf{y}) = \sqrt{3^2 + 4^2} = \sqrt{25} = 5$ (Figure **??**).

- **Minkowski Distance:** a generalization of the Euclidean distance obtained by replacing the fixed exponent 2 (both in the power and in the root) with a parameter r:

$$d(\mathbf{x}, \mathbf{y}) = \left(\sum_{k=1}^{n} |x_k - y_k|^r \right)^{1/r}$$

  It is important not to confuse r with n: n is the number of attributes (a property of the data), while r is a parameter chosen by the user that determines *how* the differences are combined. The most common cases are:

  - $r = 1$, **Manhattan** (city block, L_1 norm) distance: the sum of the absolute differences, $d = \sum_k |x_k - y_k|$. It corresponds to moving along a grid, as between city blocks.
  - $r = 2$, **Euclidean** (L_2 norm) distance.
  - $r \to \infty$, **Supremum** (Chebyshev, L_∞ norm) distance: the maximum difference between any component of the vectors, $d = \max_k |x_k - y_k|$.

  As r grows, the largest difference dominates the others. For the same points as before: $L_1 = 3 + 4 = 7$, $L_2 = 5$, $L_\infty = \max(3, 4) = 4$ (Figure **??**).

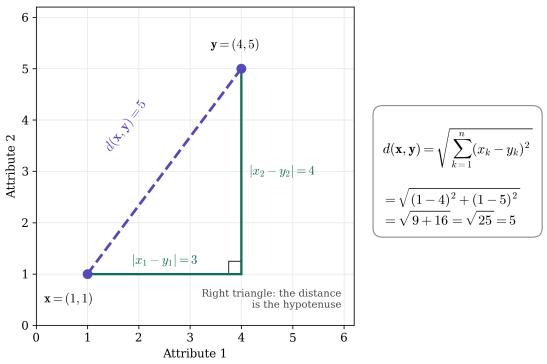


Figure 1.1: Euclidean distance between $\mathbf{x} = (1, 1)$ and $\mathbf{y} = (4, 5)$: the per-attribute differences are the legs of a right triangle and the distance is its hypotenuse.

Note: if attributes have very different scales (e.g. income in euros and age in years), the attribute with the largest range dominates the distance. For this reason, attributes are usually standardized before computing these distances.

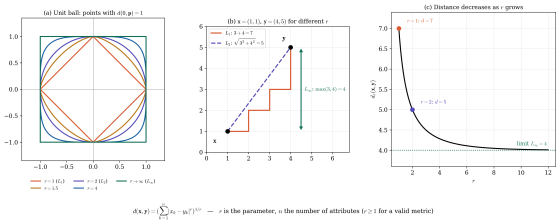


Figure 1.2: Minkowski distance. (a) Unit ball for different values of r: a diamond for L_1, a circle for L_2, a square for L_∞. (b) Distance between $\mathbf{x} = (1, 1)$ and $\mathbf{y} = (4, 5)$: $L_1 = 7$ (sum of differences), $L_2 = 5$ (straight line), $L_\infty = 4$ (maximum difference). (c) The distance decreases as r grows and converges to L_∞.

1.2.3 Properties of a Distance Metric

A distance d is called a **metric** if it satisfies the following properties for all points x, y, z of the space:

- **Identity:** the distance from a point to itself is zero: $d(x, x) = 0$.
- **Positivity:** the distance between two distinct points is strictly positive: if $x \neq y$, then $d(x, y) > 0$. Together with identity, this means that $d(x, y) = 0$ if and only if $x = y$.
- **Symmetry:** the order of the points does not matter: $d(x, y) = d(y, x)$.
- **Triangle Inequality:** going directly from x to z is never longer than passing through an intermediate point y: $d(x, z) \leq d(x, y) + d(y, z)$.

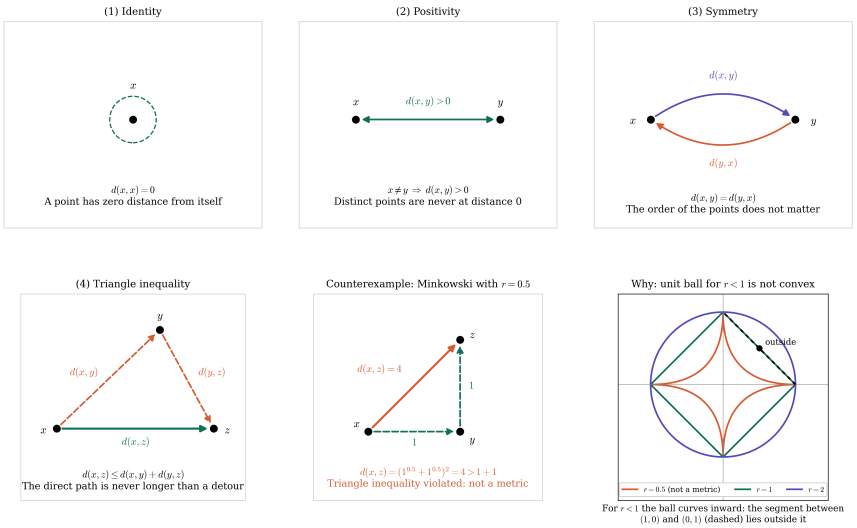


Figure 1.3: Properties of a distance metric. (1)–(4) The four properties: identity, positivity, symmetry and triangle inequality. Bottom centre: the Minkowski distance with $r = 0.5$ violates the triangle inequality, since $d(x, z) = 4 > d(x, y) + d(y, z) = 2$. Bottom right: for $r < 1$ the unit ball is not convex, which is the geometric reason why the triangle inequality fails.

The Minkowski distance is a metric for every $r \geq 1$. For $r < 1$ the triangle inequality is violated: for instance, with $r = 0.5$, $x = (0, 0)$, $z = (1, 1)$ and $y = (1, 0)$

we get $d(x, z) = (1 + 1)^2 = 4$, while $d(x, y) + d(y, z) = 1 + 1 = 2$, so the direct path would be longer than the detour.

1.2.4 Mahalanobis Distance

The Mahalanobis distance measures the distance between two points while taking into account the distribution of the data, described by its covariance matrix S:

$$d(\mathbf{x}, \mathbf{y}; S) = \sqrt{(\mathbf{x} - \mathbf{y})^\top S^{-1} (\mathbf{x} - \mathbf{y})}$$

The inverse covariance matrix S^{-1} reweights the differences between the points:

- along directions where the data have **high variance**, a difference counts *less*, because large spreads are normal there;
- along directions where the data have **low variance**, the same difference counts *more*, because it is unusual;
- **correlations** between attributes are also taken into account, so redundant information is not counted twice.

As a consequence, the Mahalanobis distance is unitless and scale-invariant. Two special cases help to interpret it:

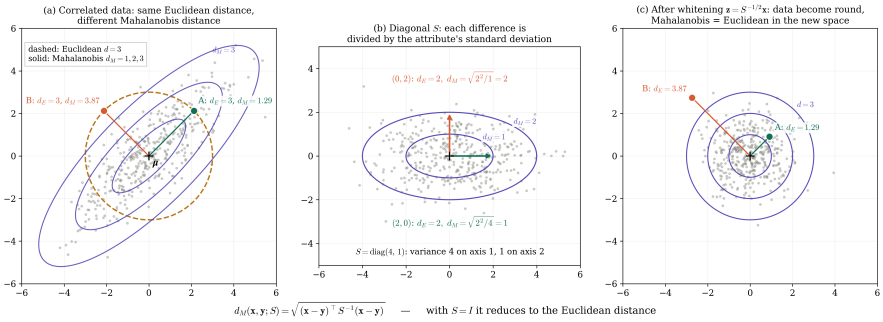


Figure 1.4: Mahalanobis distance. (a) With correlated data, points A and B have the same Euclidean distance from the mean $\boldsymbol{\mu}$, but A lies along the direction of high variance ($d_M = 1.29$) while B lies across it ($d_M = 3.87$): the contours of constant Mahalanobis distance are ellipses that follow the shape of the data. (b) With a diagonal S each difference is divided by the standard deviation of its attribute: the same Euclidean difference of 2 gives $d_M = 1$ along the high-variance axis and $d_M = 2$ along the low-variance one. (c) Transforming the data with $S^{-1/2}$ (whitening) makes the distribution round, and the Euclidean distance in the new space equals the Mahalanobis distance in the original one.

- if $S = I$ (identity matrix: unit variances and no correlation), it reduces to the **Euclidean distance**;
- if S is diagonal, it is equivalent to the Euclidean distance computed on **standardized** attributes, i.e. each difference is divided by the standard deviation of its attribute.

Example: let $S = \begin{pmatrix} 4 & 0 \\ 0 & 1 \end{pmatrix}$, so the first attribute has variance 4 and the second variance 1. For a difference $\mathbf{x} - \mathbf{y} = (2, 0)$ we get $d = \sqrt{2^2/4} = 1$, while for $\mathbf{x} - \mathbf{y} = (0, 2)$ we get $d = \sqrt{2^2/1} = 2$. Both pairs have Euclidean distance 2, but the Mahalanobis distance considers the second pair farther apart, since a difference of 2 is much less common along the second attribute.

1.3 Data PreProcessing

Real-world data are rarely ready to be analysed: they can be noisy, redundant, too large, described by too many attributes, or stored in formats that algorithms cannot handle (e.g. text). **Data PreProcessing** is the set of techniques that turn raw data into a form that is cleaner, smaller and more suitable for the analysis. The main techniques are:

- **Data Aggregation:** combining two or more attributes or objects into a single one.
- **Data Reduction:** obtaining a smaller dataset that gives (almost) the same analytical results.
- **Feature Selection:** keeping only the useful attributes, needed to tackle the *curse of dimensionality* (Section **??**).
- **Feature Transformation or Normalization:** changing the values or the scale of attributes (Sections **??**–**??**).
- **Preprocessing for textual data:** turning documents into structured data (Section **??**).

Data Aggregation. Aggregation replaces a group of objects (or attributes) with a single one that summarises them, typically with a sum or an average. Examples: daily sales → monthly sales, cities → regions, single transactions → total per customer. It is used for three purposes:

- **data reduction:** fewer objects to store and process (365 days become 12 months);
- **change of scale:** looking at the data at a coarser level of detail (regions instead of cities);
- **more stable data:** averages vary less than single values, because random fluctuations tend to cancel out (Figure **??**).

The price to pay is the loss of detail: after aggregating into months, it is no longer possible to know what happened on a specific day.

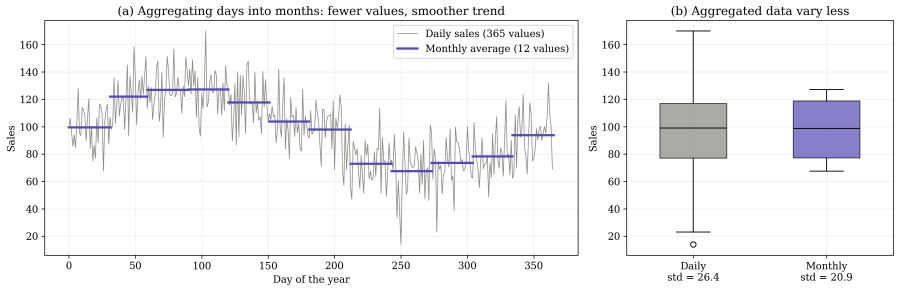


Figure 1.5: Data aggregation. (a) 365 daily values are replaced by 12 monthly averages, which show the trend more clearly. (b) The aggregated values have a lower variability (smaller standard deviation) than the original ones.

Data Reduction. Data reduction generates a reduced representation of the dataset, *smaller in volume but able to provide similar analytical results.* A dataset is a table, so it can be reduced along three directions:

- **fewer rows**, i.e. fewer objects: *sampling* (reduces the cardinality of the dataset);
- **fewer columns**, i.e. fewer attributes: *feature selection and creation*;
- **fewer distinct values per column**: *discretization* (reduces the cardinality of the attribute domains, e.g. age $\rightarrow$ {young, adult, senior}).

Sampling. Sampling is the main technique for data selection and reduction: instead of analysing all the objects, only a subset of them (the *sample*) is analysed. It is useful when processing the whole dataset is too expensive or too slow. It is based on two key principles:

- using a sample works almost as well as using the entire dataset, *if the sample is representative*;
- a sample is **representative** if it has approximately the same property of interest as the original dataset (e.g. the same mean, or the same proportion of each class).

There are two main types of sampling:

- **Simple Random Sampling:** every object has the same probability of being selected. It is simple, but with small samples or rare classes it may produce an unrepresentative sample by chance: a class that is 10% of the data could be missing entirely (Figure **??**b).
- **Stratified Sampling:** the data are first split into groups (*strata*), for instance one per class, and then a random sample is drawn from each group, proportionally to its size. This guarantees that every group is represented with the right proportion (Figure **??**c).

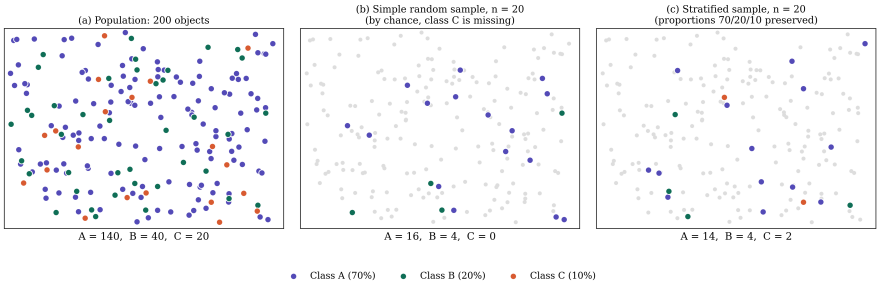


Figure 1.6: Sampling 20 objects from a population of 200 with imbalanced classes (70%, 20%, 10%). Simple random sampling may miss the rarest class, while stratified sampling preserves the original proportions.

1.3.1 Curse of Dimensionality

The **curse of dimensionality** is the set of problems that arise when the number of attributes (dimensions) becomes large. The key idea is simple: *when the number of dimensions increases, the volume of the space grows so fast that the data become sparse*, i.e. points are far from each other and the space is mostly empty.

Two effects make this concrete (Figure **??**):

- **Neighbourhoods are no longer local.** With uniform data in a unit cube, a sub-cube that contains 10% of the points has edge 0.1 in 1 dimension, but edge 0.79 in 10 dimensions: to find "nearby" points one must cover almost the whole range of every attribute.
- **Distances become all similar.** As the dimensions grow, the distance to the nearest point and the distance to the farthest point become almost the same. If all points are roughly equally distant, the concept of "near" loses its meaning.

As a consequence, the definitions of **density** and **distance** between points, which are critical for clustering and outlier detection, become less meaningful.

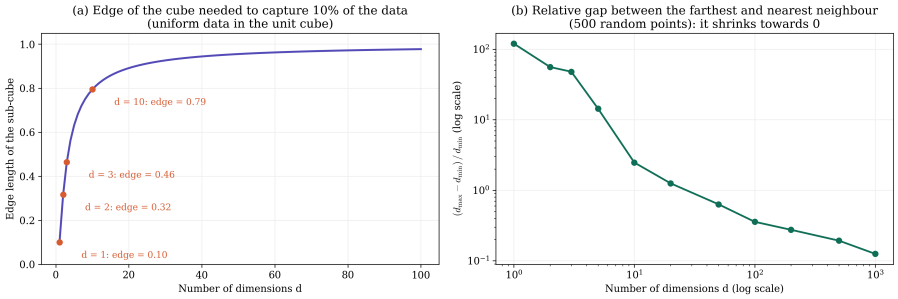


Figure 1.7: Curse of dimensionality. (a) To capture 10% of uniformly distributed data, the edge of the sub-cube quickly approaches the full range as the number of dimensions grows. (b) The relative difference between the farthest and the nearest neighbour shrinks towards zero: in high dimensions all points look equally distant.

Feature Subset Selection. One way to counter the curse of dimensionality is to keep only a subset of the attributes. Feature Subset Selection reduces the dimensionality of the data by removing:

- **redundant features**, which duplicate much or all of the information contained in other attributes. Example: the price of a product with and without VAT; knowing one, the other adds nothing.
- **irrelevant features**, which contain no useful information for the task at hand. Example: the student ID when predicting the student's grade.

Recursive Feature Elimination (RFE). A common way to perform feature selection is to remove features one step at a time, always discarding the least useful one (Figure **??**):

1. assign a weight (importance) to each feature, for instance the absolute value of the coefficients of a linear model;
2. train the model (*estimator*) on the current set of features and obtain the importance of each feature;
3. remove (*prune*) the least important feature(s) from the current set;
4. repeat steps 2–3 on the reduced set, until the desired number of features is reached.

The model is retrained at every iteration because removing a feature can change the importance of the others. For the same reason, RFE can be expensive when there are many features.

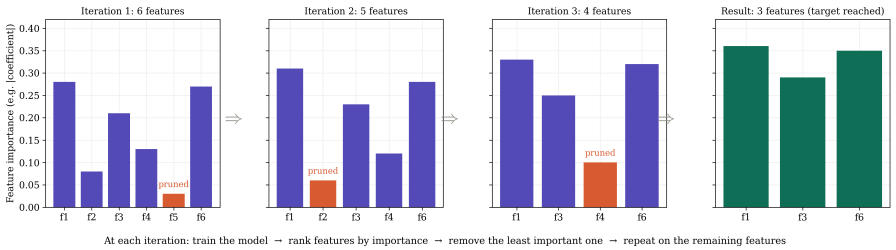


Figure 1.8: Recursive Feature Elimination with a target of 3 features. At each iteration the model is retrained, the importances are recomputed and the least important feature (in orange) is removed.

Other approaches.

- **Feature importance of interpretable models:** some models directly tell how much each feature contributes to the prediction, for instance Decision Trees (how much each feature is used to split the data) and Linear Regression (the size of the coefficients, if the features are on the same scale). The least important features can then be discarded.
- **Decomposition techniques** such as PCA and SVD can be used to reduce dimensionality automatically. Strictly speaking, they do not select original features but *create new ones*: each component is a combination of the original attributes, and the first few components capture most of the information in the data. Looking at how much each original attribute contributes to the main components also tells which attributes are the most important.

1.3.2 Feature Engineering

Feature Engineering is the act of extracting features from raw data and transforming them into formats that are more suitable for a Machine Learning model. A good representation of the data often matters more than the choice of the algorithm. Different techniques are needed according to the type of data:

Data type	Examples	Typical techniques
Structured	numerical, categorical attributes	normalization, discretization, binarization (one-hot encoding)
Unstructured	text, images, signals	tokenization and term vectors, Fourier transform, extraction of descriptors
Mixed	e.g. a product with price, category and description	combination of the above

The basic Feature Engineering techniques, described in the next subsections, are **data transformation**, **discretization**, **binarization** and **normalization**.

1.3.3 Data Transformation

Data transformation is the process of converting data from one format or representation to another, so that the important information is captured much more efficiently than in the original representation.

The classic example is the **Fourier Transform**, which converts a signal from the *time domain* (value at each instant) to the *frequency domain* (which frequencies the signal contains, and how strong they are). A signal made of a few overlapping waves looks complicated as a sequence of values, but in the frequency domain it is summarised by a few peaks (Figure **??**).

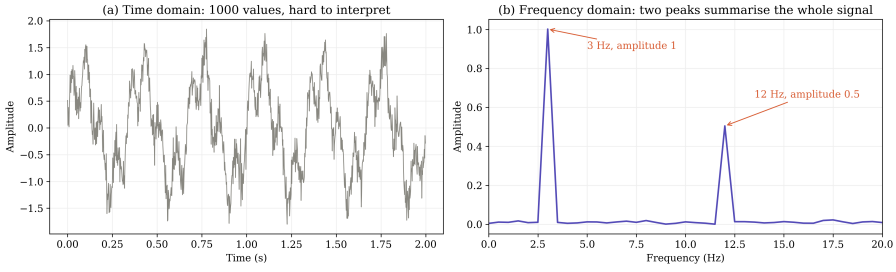


Figure 1.9: Fourier transform. (a) A noisy signal made of two waves (3 Hz and 12 Hz), represented by 1000 values over time. (b) The same signal in the frequency domain: two peaks describe it completely.

Another very common case is the conversion of **non-numerical data into numerical data**: most algorithms work only with numbers, so categories and text must be transformed (see Sections **??** and **??**).

1.3.4 Data Discretization

Discretization converts a **continuous** attribute into an **ordinal** one: a potentially infinite number of values is mapped into a small number of ordered categories (*bins*). Example: age $\in [15, 90] \rightarrow$ {young, adult, senior}. It is useful because some algorithms only accept categorical data, and because categories are often easier to interpret. The price is a loss of detail: ages 36 and 55 may end up in the same bin.

The main **unsupervised** discretization techniques (they do not use class labels) are (Figure **??**):

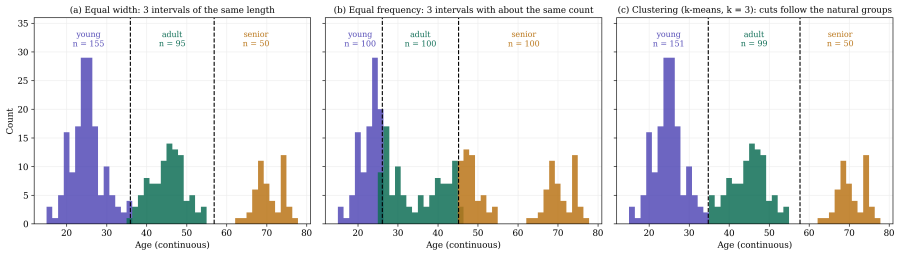


Figure 1.10: Discretization of a continuous attribute (age) into 3 bins with three unsupervised techniques. The dashed lines are the cut points; each colour is one category.

- **Equal width:** N intervals of the same length. Simple, but some bins may contain very few points, and it is sensitive to outliers.
- **Equal frequency:** N intervals with approximately the same number of points. Each bin is well populated, but similar values may be split into different bins.
- **Clustering:** the cuts are placed between natural groups of values found by a clustering algorithm (e.g. k-means).

1.3.5 Data Binarization

Binarization maps an attribute into one or more **binary** variables (0/1).

- **Continuous attribute:** first discretize it into categories, then apply one of the techniques for categorical attributes below. Example: age $\rightarrow$ {young, adult, senior} $\rightarrow$ binary vector.
- **Categorical attribute:** map it to a set of binary attributes, for instance via *one-hot encoding*.

Why not simply map the categories to integers (San Francisco = 1, New York = 2, Seattle = 3)? Because the model would assume an order and a distance that do not exist (Seattle would be "greater" than San Francisco, and New York would be "halfway" between them). The following encodings avoid this problem. In all of them, a categorical variable with k categories is represented by a group of bits.

- **One-Hot Encoding:** one bit per category (k bits). Since an object belongs to only one category, exactly one bit is set to 1. The result is a *sparse* matrix (mostly zeros).

- **Dummy Coding:** only $k-1$ bits. One category is chosen as *reference* and is encoded with all zeros; the other categories are encoded as in one-hot.
- **Effect Coding:** like dummy coding, but the reference category is encoded with all -1s instead of all 0s.

	One-Hot			**Dummy**		**Effect**	
	e_1	e_2	e_3	e_1	e_2	e_1	e_2
San Francisco	1	0	0	1	0	1	0
New York	0	1	0	0	1	0	1
Seattle (reference)	0	0	1	0	0	-1	-1

Why $k-1$ bits are enough. One-hot encoding uses k bits, i.e. it has k degrees of freedom, but the variable only needs $k-1$. In the example, if $e_1 = 0$ and $e_2 = 0$, the city *must* be Seattle, so e_3 adds no information: it can always be computed as $e_3 = 1 - e_1 - e_2$. This redundancy is a problem for linear models, because the model can no longer find a unique set of coefficients (many combinations give exactly the same predictions). Dummy and effect coding remove the extra bit.

Pros and cons. When these encodings are used in a linear model, they also change how the coefficients are interpreted:

	PROS	**CONS**
One-Hot	each feature clearly corresponds to one category; missing data can be encoded as the all-zeros vector; the intercept of a linear model represents the overall mean of the target variable	redundant (k bits for $k-1$ degrees of freedom): the linear model has no unique solution
Dummy	not redundant ($k-1$ bits): the solution is unique; the intercept is the mean of the target for the reference category	cannot easily handle missing data, since the all-zeros vector is already used by the reference category
Effect	not redundant; the reference category uses a different code (-1s), so the all-zeros vector is free for missing data; the intercept represents the overall mean	the vector of all -1s is dense (not sparse), which is expensive for both storage and computation

1.3.6 Attribute Transformation

An attribute transformation is a function that maps *all* the values of an attribute to a new set of values, so that each old value corresponds to exactly one new value. Examples: $x \to \log x$ (to compress very skewed values), $x \to |x|$, and normalization.

Normalization is used to make attributes with different scales comparable. If age ranges in $[20, 70]$ and income in $[20\,000, 100\,000]$, any distance between two people would depend almost only on income, simply because its numbers are bigger. After normalization all attributes contribute in a comparable way. The two most common techniques are:

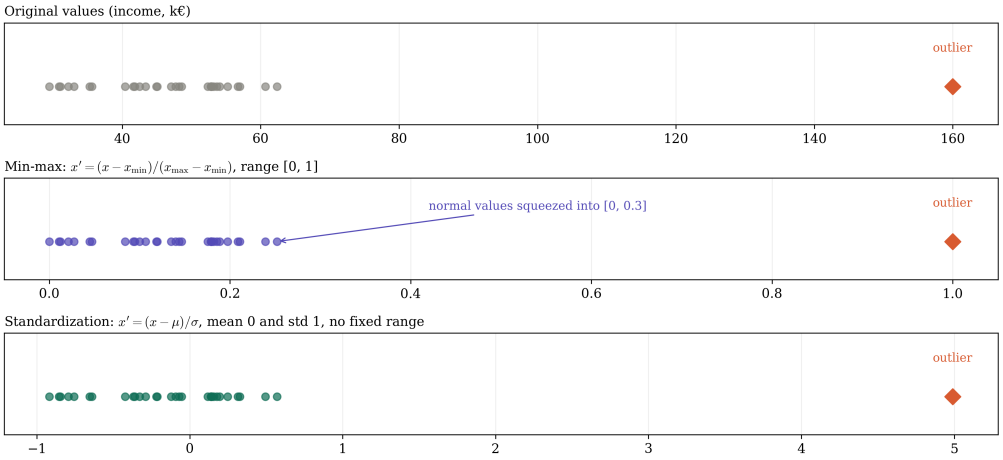


Figure 1.11: Effect of normalization on data with an outlier. Min-max normalization maps the outlier to 1 and compresses all the normal values close to 0; standardization centres the data around 0 without a fixed range.

- **Min-Max Normalization:** rescales the values linearly into the range $[0, 1]$: the minimum becomes 0 and the maximum becomes 1.

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

  It retains the shape of the distribution, but it is **sensitive to outliers**: a single extreme value becomes the maximum and squeezes all the other values into a small part of the range (Figure **??**). Example: with $x_{\min} = 20$ and $x_{\max} = 70$, the value $x = 45$ becomes $x' = 25/50 = 0.5$.

- **Standardization (z-score):** subtracts the mean μ and divides by the standard deviation σ, so that the new values have zero mean and unit standard deviation.

$$x' = \frac{x - \mu}{\sigma}$$

  The new value tells *how many standard deviations* x is from the mean. It is not bounded to a fixed range, and outliers remain clearly visible as large values. Example: with $\mu = 50$ and $\sigma = 10$, the value $x = 70$ becomes $x' = 2$.

Note: the parameters ($x_{\min}$, $x_{\max}$, μ, σ) must be computed on the training data only, and then applied unchanged to the test data.

1.3.7 Data Preparation for Document Data

Most algorithms cannot process raw text directly: they need numbers. For this reason a document is usually represented as a set of features. **Document PreProcessing** is the activity that generates a structured representation of documents, and it includes five sequential steps:

1. **Document Splitting:** depending on the goal of the analysis, documents are split into sentences or paragraphs, or analysed in their entire content.

2. **Tokenization:** breaking the text into units called *tokens*, typically words, for instance by separating words at spaces and identifying sentence boundaries based on punctuation.

3. **Case Normalization:** converting every token to lower-case (or upper-case), so that "The" and "the" are treated as the same word.

4. **Stopword Elimination:** removing *stopwords*, the most common words of a language (articles, prepositions, auxiliary verbs, ...), which appear everywhere and carry little meaning.

5. **Stemming:** reducing each word to its root (*stem*), so that different forms of the same word are counted together: "running", "runs" → "run"; "cats" → "cat".

The following example shows how one sentence changes at each step:

Step	**Result**
Original text	`The cats are running in the garden. The dog runs and the dog barks.`
1. Splitting	S1: `The cats are running in the garden.` S2: `The dog runs and the dog barks.`
2. Tokenization (S1)	`[The, cats, are, running, in, the, garden]`
3. Case normalization	`[the, cats, are, running, in, the, garden]`
4. Stopword elimination	`[cats, running, garden]`
5. Stemming	`[cat, run, garden]`

Term vectors (bag of words). After preprocessing, each document is represented as a **term vector**: each distinct term is an attribute, and the value of the attribute is the number of times the term occurs in the document. The order of the words is ignored. Putting all the vectors together gives the **document-term matrix**. Example with three short documents:

- D_1: "The cats are running in the garden" → {cat, run, garden}
- D_2: "The dog runs and the dog barks" → {dog, run, dog, bark}
- D_3: "A dog and a cat in the garden" → {dog, cat, garden}

	bark	**cat**	**dog**	**garden**	**run**
D_1	0	1	0	1	1
D_2	1	0	2	0	1
D_3	0	1	1	1	0

Weighting schemes. The value of each term can be computed in different ways:

- **Binary:** 1 if the term appears in the document, 0 otherwise.
- **Term Frequency (TF):** the number of occurrences of the term in the document (as in the table above).
- **TF-IDF:** the term frequency multiplied by a factor that rewards rare terms and penalises terms that appear in many documents:

$$\text{tf-idf}(t, d) = \text{tf}(t, d) \cdot \log \frac{N}{\text{df}(t)}$$

  where N is the number of documents and $\text{df}(t)$ is the number of documents containing term t. A term that appears in every document gets weight $\log 1 = 0$, because it does not help to distinguish documents. In the example ($N = 3$, natural logarithm), "bark" appears only in D_2, so its weight is $1 \cdot \log 3 \approx 1.10$, while "dog" appears in two documents and gets $2 \cdot \log 1.5 \approx 0.81$ in D_2, even though it occurs twice.